CSA4707-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 2: Covariance, Eigenvalues and Eigenvectors Analysis

Problem Statement: A hospital lab records several correlated blood parameters per patient. Analysts must know which parameters move together and which directions carry most of the variation. A dataset of 200 patient records with haemoglobin, RBC count, glucose, cholesterol and BMI. The lab suspects redundancy among these readings. The objective is to compute covariance and correlation, then extract eigenvalues and eigenvectors to find the principal directions of variation. Use NumPy, Pandas, Matplotlib.

Tasks:

1. Load the data into Pandas and standardise each feature.
2. Compute the covariance and correlation matrices; plot them as heatmaps.
3. Compute eigenvalues and eigenvectors with NumPy; sort by eigenvalue.
4. Plot the data in 2-D with the top two eigenvectors drawn as arrows.
5. Report the percentage of variance captured by each eigenvalue.

FORMULA:

1. Standardisation

$$Z = \frac{X-\mu}{\sigma}$$

2. Covariance

Cov(X, Y) = Sum((X_i - X_bar) * (Y_i - Y_bar)) / (n - 1)

3. Correlation

r_XY = Cov(X, Y) / (sigma_X * sigma_Y)

4. Eigenvalue and Eigenvector

$$Cv = \lambda v$$

$$\det(C - \lambda I) = 0$$

5. Percentage of Variance

$$VariancePercentage = \frac{\lambda_i}{\sum_{j=1}^{p} \lambda_j} \times 100$$

PROGRAM CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(10)

data = pd.DataFrame({
    "Haemoglobin": np.random.normal(13, 2, 200),
    "RBC": np.random.normal(5, 0.5, 200),
    "Glucose": np.random.normal(100, 15, 200),
    "Cholesterol": np.random.normal(200, 25, 200),
    "BMI": np.random.normal(25, 4, 200)
})

X = (data - data.mean()) / data.std()

cov = X.cov()
corr = X.corr()

plt.figure(figsize=(6, 4))
plt.imshow(cov, cmap="coolwarm")
plt.colorbar()
plt.xticks(range(5), data.columns, rotation=45)
```

```
plt.yticks(range(5), data.columns)
plt.title("Covariance Matrix")
plt.tight_layout()
plt.show()

plt.figure(figsize=(6, 4))
plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar()
plt.xticks(range(5), data.columns, rotation=45)
plt.yticks(range(5), data.columns)
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()

values, vectors = np.linalg.eigh(cov)
order = np.argsort(values)[::-1]
values = values[order]
vectors = vectors[:, order]

print("Eigenvalues:", values)
print("Eigenvectors:\n", vectors)

plt.figure(figsize=(6, 4))
plt.scatter(X.iloc[:, 0], X.iloc[:, 1])

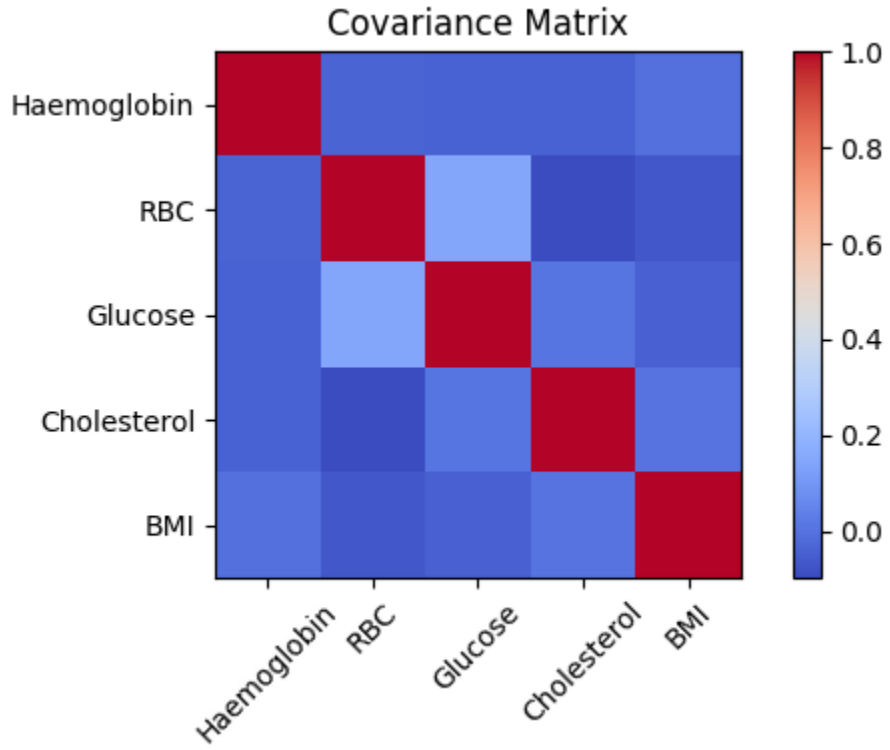
for i in range(2):
    plt.arrow(0, 0, vectors[0, i] * 3, vectors[1, i] * 3,
              head_width=0.15, length_includes_head=True)

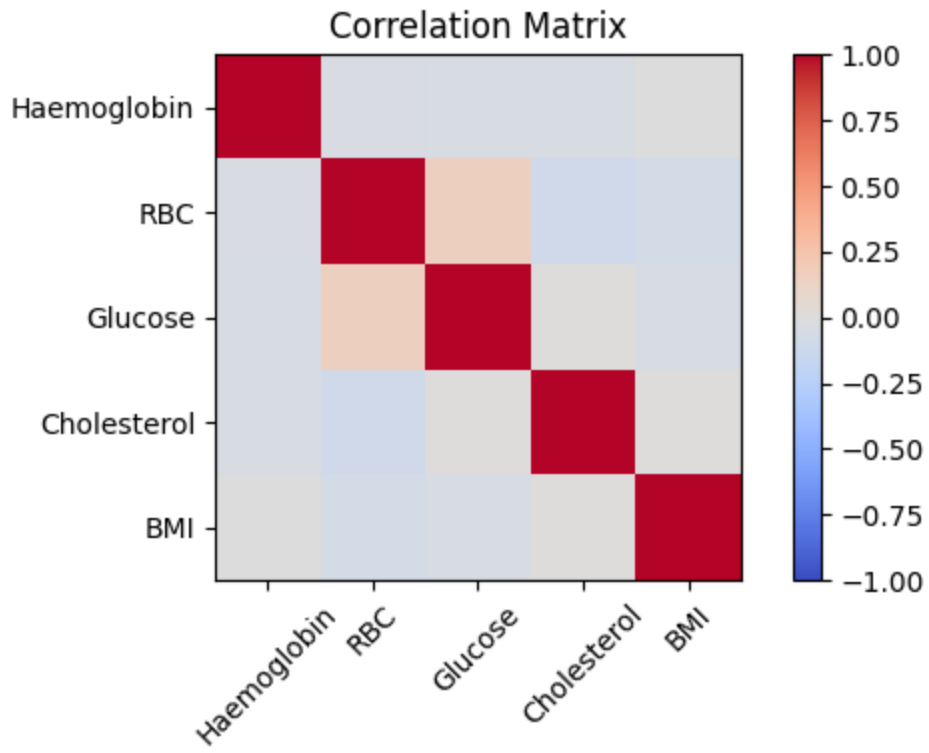
plt.xlabel("Haemoglobin")
plt.ylabel("RBC")
```

```
plt.title("Top Two Eigenvectors")
plt.grid()
plt.show()

variance = values / values.sum() * 100
print("Variance Percentage:")
print(np.round(variance, 2))
```

OUTPUT:





Eigenvalues: [1.20878371 1.05524414 0.99055638 0.93098591 0.81442986]

Eigenvectors:

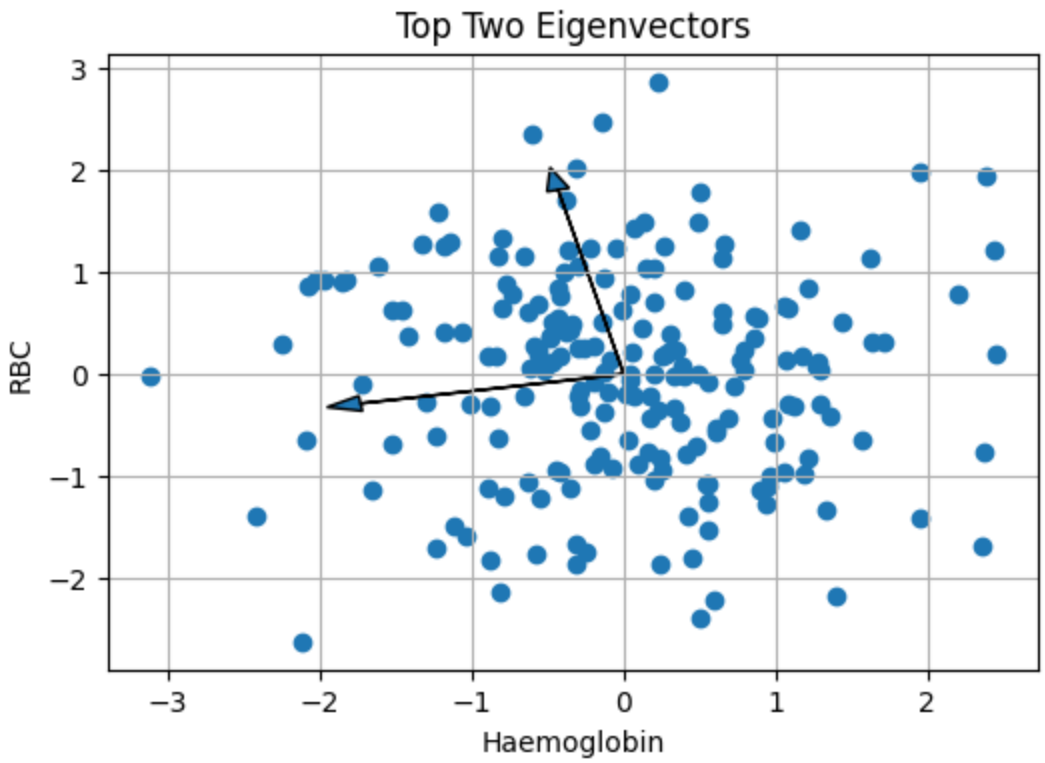
[[-0.16169148 -0.65157583 -0.41616488 0.60397665 0.10641333]

[0.673023 -0.10807503 0.14953751 0.04063175 0.71508561]

[0.58006448 0.27212576 0.00687819 0.54828601 -0.53740852]

[-0.26907561 0.69976708 -0.37466043 0.3511832 0.41740181]

[-0.33469303 0.00650599 0.81487887 0.45783631 0.11956833]]



Variance Percentage:

[24.18 21.1 19.81 18.62 16.29]